

Laboratório 2 - Unidade Operativa Uniciclo RISC-V32I

1st Artur Henrique do Carmo Rosa
Departamento de Ciências da Computação
Universidade de Brasília
Brasília, Brazil
222028738@aluno.unb.br

2nd Daniel Dezan Baby
Departamento de Ciências da Computação
Universidade de Brasília
Brasília, Brazil
232046097@aluno.unb.br

3rd João Filipe Araújo
Departamento de Ciências da Computação
Universidade de Brasília
Brasília, Brazil
231013574@aluno.unb.br

4th Leonardo Tomé Sampaio
Departamento de Ciências da Computação
Universidade de Brasília
Brasília, Brazil
222005448@aluno.unb.br

5th Luís Eduardo Curi Serra
Departamento de Ciências da Computação
Universidade de Brasília
Brasília, Brazil
251033574@aluno.unb.br

Abstract—No presente trabalho é apresentado o desenvolvimento da Unidade Operativa Uniciclo da arquitetura RISC-V32I, implementada na plataforma Quartus. O projeto contempla os módulos que compõem o caminho de dados (*datapath*), as memórias de instrução e de dados em organização Harvard, a unidade de controle principal e a unidade de controle da ULA, de forma a executar automaticamente um conjunto de instruções de 32 bits pré-compiladas no formato MIF. Além do conjunto básico de referência da literatura (Tipo-R, *lw*, *sw* e *beq*), foram implementadas as instruções de salto e de formação de constantes (*jal*, *jalr*, *lui* e *auipc*), o que exigiu a extensão do caminho de dados clássico com novos sinais de controle e novos multiplexadores. O trabalho inclui a descrição da arquitetura desenvolvida, do conjunto de instruções suportado, dos procedimentos e métodos adotados, dos resultados obtidos por meio do ambiente de simulação da forma de onda, seguidos de uma análise crítica dos resultados e das principais dificuldades de implementação e depuração enfrentadas, em especial nas instruções de salto e no tratamento de endereços fora da região válida da memória de instruções.

Index Terms—RISC-V, Uniciclo, FPGA, Quartus, caminho de dados, unidade de controle

I. DIVISÃO DE TAREFAS

A. Objetivos e Introdução (Teoria da Arquitetura)

Seção ii. Objetivos: resumo sucinto dos objetivos do laboratório. Seção iii. Introdução (parte 1): visão geral da arquitetura RISC-V32I, filosofia RISC, organização Uniciclo e caminho de dados.

B. Introdução (Caminho de Dados e Memórias)

Seção iii. Introdução (parte 2): detalhamento dos módulos do caminho de dados, memórias de instrução e dados em

organização Harvard, banco de registradores, somador, gerador de imediato e ULA.

C. Introdução (Unidade de Controle e Multiplexadores)

Seção iii. Introdução (parte 3): unidade de controle principal, unidade de controle da ULA, sinais de controle e multiplexadores do caminho de dados.

D. Procedimentos e Métodos

Seção iv. Materiais e Métodos: ferramentas utilizadas (Quartus II, Waveform Editor), descrição da implementação de cada módulo e metodologia de sincronização e integração.

E. Resultados experimentais

Seção v. Resultados: simulações no Waveform Editor, conteúdo dos registradores, sinais de controle e análise de desempenho de cada unidade funcional.

F. Análise de resultados, Conclusões e Revisão Geral

Seção vi. Discussão e Conclusões: análise qualitativa e quantitativa, gargalos identificados, pontos críticos, dificuldades de implementação e depuração, e conclusão geral do laboratório.

II. OBJETIVOS

O objetivo deste laboratório foi desenvolver a Unidade Operativa Uniciclo da arquitetura RISC-V32I, utilizando o ambiente Quartus II Altera, de forma a montar os módulos principais do caminho de dados (*datapath*) capazes de processar instruções de 32 bits pré-compiladas no formato MIF, fazendo uso de memórias de instrução e de dados em organização

Harvard. De forma específica, buscou-se estender o modelo de referência apresentado na literatura [2] para dar suporte às instruções de salto incondicional (*jal*, *jalr*) e às instruções de formação de constantes (*lui*, *auipc*), implementando a inclusão de novos sinais de controle e novos caminhos de dados na microarquitetura.

III. INTRODUÇÃO

A. Arquitetura RISC-V32I Uniciclo

A arquitetura RISC-V32I segue a filosofia RISC (*Reduced Instruction Set Computer*), caracterizada por um conjunto reduzido e regular de instruções de tamanho fixo (32 bits), esse conjunto facilita a decodificação e o projeto do caminho de dados, pois os campos da instrução (*opcode*, *funct3*, *funct7*, registradores e imediato) ocupam posições previsíveis em cada formato.

Na organização Uniciclo, toda instrução é integralmente buscada, decodificada e executada dentro de um único ciclo de *clock*. Não há sobreposição entre estágios, de modo que o período do *clock* é dimensionado para acomodar o pior caminho crítico, o caminho da instrução de carga (*lw*), que percorre memória de instruções, banco de registradores, ULA e memória de dados e escrita no banco de registradores, [2]. Como consequência, a simplicidade de controle do modelo Uniciclo é obtida ao custo de um período de *clock* ditado pela instrução mais lenta.

O projeto adota organização de memória do tipo Harvard, com memória de instruções e memória de dados fisicamente separadas, cada uma com seu próprio barramento de endereços e de dados. Essa separação permite que a busca da instrução e o eventual acesso ao dado ocorram no mesmo ciclo, sem conflito de acesso a uma única memória. Os componentes que compoem o caminho de dados são implementados em módulos segregados com entradas e saídas bem definidas, os quais são posteriormente ligados para implementar o processador final.

B. Contador de Programa e Memória de Instruções

O contador de programa (PC) é um registrador de 32 bits que armazena o endereço da instrução em execução. Trata-se do único elemento de entrada da memória de instruções. A cada borda de subida do *clock*, o PC é atualizado com o endereço da próxima instrução, calculado de forma combinacional pelo restante do caminho de dados. O módulo possui ainda uma entrada de *reset* assíncrono que, quando ativada, inicializa o PC com o endereço base, `0x00004000`.

A memória de instruções é somente de leitura e é inicializada pelo conteúdo do arquivo `UnicicloInst.mif`. Como o PC trabalha com endereços de *byte* a partir de `0x00004000`, e a memória é organizada em palavras de 32 bits, é necessário traduzir o endereço recebido em um índice de palavra: subtrai-se o endereço base (`TEXT_BASE`) do endereço corrente e divide-se o resultado por quatro, deslocando dois bits a direita. Além disso, para impedir acesso a endereços indevidos a memória verifica se o endereço solicitado pertence à faixa válida do segmento de texto.

Esse mecanismo de validação é discutido em detalhe na Seção VI-A.

C. Banco de Registradores

O banco de registradores implementa os 32 registradores de 32 bits do RISC-V32I (*x0* a *x31*). Ele dispõe de duas portas de leitura assíncronas, combinacionais, endereçadas pelos campos *rs1* e *rs2* da instrução, e de uma porta de escrita síncrona, endereçada pelo campo *rd* e habilitada pelo sinal de controle *RegWrite*. A escrita ocorre na borda de subida do *clock*, garantindo que o valor lido em um ciclo corresponda ao conteúdo anterior à escrita do mesmo ciclo. O registrador *x0* é mantido permanentemente em zero. O módulo expõe o conteúdo de todos os 32 registradores em saídas dedicadas, possibilitando a visualização simultânea de toda a janela de registradores na simulação.

D. Unidade Lógica e Aritmética (ULA) e seu Controle

A Unidade Lógica e Aritmética (ULA) é o bloco combinacional responsável pelas operações aritméticas e lógicas do processador. Em sua entrada recebe dois operandos de 32 bits e um código de operação de 4 bits; em sua saída produz o resultado de 32 bits e o sinal *Zero*, que indica se o resultado é nulo e é utilizado pela lógica de desvio condicional. As operações implementadas incluem soma, subtração, AND, OR, XOR, deslocamento lógico à esquerda (*SLL*), deslocamento lógico à direita (*SRL*), comparação com sinal (*SLT*) e uma operação de repasse do segundo operando (*PASSA-B*), introduzida especialmente para dar suporte à instrução *lui*.

A escolha da operação efetiva fica a cargo da unidade de controle da ULA (*ALU control*), um bloco combinacional que combina o sinal *ALUOp*, gerado pela unidade de controle principal, com os campos *funct3* e o bit 30 da instrução (que distingue, por exemplo, *add* de *sub*). Esse arranjo em dois níveis evita que a unidade de controle principal precise conhecer todos os detalhes de cada operação: ela apenas indica a categoria (acesso à memória, desvio, Tipo-R, Tipo-I ou *lui*), e a unidade de controle da ULA refina essa categoria na operação final [2].

E. Somador

O caminho de dados utiliza somadores dedicados de 32 bits, descritos como blocos puramente combinacionais que produzem a soma de duas entradas. São empregados dois somadores no estágio de atualização do PC: o primeiro calcula $PC + 4$, endereço da instrução sequencial seguinte; o segundo calcula $PC + \text{imediato}$, endereço alvo dos desvios condicionais (*beq*, *bne*) e do salto *jal*. Manter esses somadores como módulos separados e simples mantém o caminho de dados próximo do modelo de referência e facilita o rastreamento dos endereços na simulação.

F. Gerador de Imediato

O gerador de imediato (*Imm Gen*) extrai e monta o valor imediato de 32 bits a partir dos campos da instrução, selecionando o formato correto com base no opcode. A arquitetura RISC-V32I codifica os imediatos em diferentes formatos (I, S, B, U e J), nos quais os bits do imediato aparecem espalhados em posições distintas da palavra de instrução, sempre de modo a manter o bit de sinal na posição mais significativa [1]. O módulo realiza a extensão de sinal para 32 bits em todos os formatos com sinal e, nos formatos de desvio (B) e de salto (J), reintroduz o bit menos significativo igual a zero, refletindo o fato de que esses deslocamentos são múltiplos de dois *bytes*. A correta montagem desses 32 bits é um ponto sensível do projeto e está diretamente associada às dificuldades relatadas na Seção VI-A, uma vez que um imediato mal formado leva ao cálculo de endereços de desvio incorretos.

G. Memória de Dados

A memória de dados armazena os operandos manipulados pelas instruções de carga (*lw*, *lhu*) e armazenamento (*sw*), sendo inicializada a partir do arquivo *UnicicloData.mif*. Ela é endereçada pelo resultado da ULA (que, nessas instruções, corresponde ao endereço efetivo $rs1 + \text{imediato}$) e controlada pelos sinais *MemRead* e *MemWrite*. Para compatibilizar a leitura e a escrita dentro de um único ciclo de *clock*, adotou-se a estratégia de escrever na memória na borda de subida e disponibilizar o dado lido na borda de descida do *clock*, de modo que a leitura e a escrita não disputem o mesmo instante de tempo e o caminho de dados permaneça coerente em uma execução *Uniciclo*.

H. Unidade de Controle Principal

A Unidade de Controle *Uniciclo* é o bloco responsável por observar o campo opcode da instrução, com 7 bits de largura, e decidir quais sinais devem ser ativados no restante do caminho de dados para que aquela instrução específica seja executada corretamente [2]. Diferente de um registrador ou de uma memória, ela não guarda nenhum valor de um ciclo para o outro: assim que a memória de instruções entrega um novo opcode na sua entrada, a saída já reflete os novos sinais de controle, respeitando apenas o tempo de propagação da lógica combinacional utilizada na sua implementação.

Os sinais produzidos por esse bloco, junto com o efeito de cada um sobre o caminho de dados, são os seguintes:

- *RegWrite*: habilita a escrita no banco de registradores;
- *MemRead*: habilita a leitura da memória de dados;
- *MemWrite*: habilita a escrita na memória de dados;
- *MemtoReg*: seleciona, por meio de multiplexadores, a origem do valor gravado no registrador de destino; foi ampliado para 2 bits a fim de incluir, além do resultado da ULA e do dado lido da memória, o endereço de retorno $PC + 4$ usado por *jal* e *jalr*;
- *ALUSrc*: seleciona, por meio de um multiplexador, se a segunda entrada da ULA é o conteúdo do segundo registrador lido ou o valor imediato gerado pelo gerador de imediato;

TABLE I

TABELA VERDADE REDUZIDA DA UNIDADE DE CONTROLE (MODELO DE REFERÊNCIA)

Instr.	Opcode	ASrc	M2Reg	RegW	MemR	MemW	Br	ALUOp
Tipo-R	0110011	0	0	1	0	0	0	10
lw	0000011	1	1	1	1	0	0	00
sw	0100011	1	X	0	0	1	0	00
beq	1100011	0	X	0	0	0	1	01

- *Branch*: indica que a instrução é um desvio condicional, sinal que é combinado com a saída Zero da ULA (e com o campo *funct3*) para decidir o próximo valor do contador de programa;
- *ALUOp*: indica de forma resumida o tipo de operação que a ULA deve realizar, sendo depois combinado com os campos *funct3* e *funct7* pela unidade de controle da ULA para definir a operação final;
- *Jump*, *JumpR* e *PCtoALU*: sinais adicionais, ausentes no modelo de referência, introduzidos para dar suporte às instruções *jal*, *jalr* e *auipc*, respectivamente, conforme detalhado na Seção VI-A.

A Tabela I apresenta a tabela verdade reduzida da unidade de controle para o modelo de referência, relacionando cada grupo de instruções, identificado por seu opcode, aos valores dos sinais de controle gerados [2]. O valor X (*don't care*) indica situações em que o sinal não influencia o resultado da instrução, pois seu efeito não é utilizado naquele caso. No modelo clássico, o campo *ALUOp* utiliza 2 bits para identificar a categoria da operação a ser realizada pela ULA: 00 para acessos à memória, 01 para desvios e 10 para operações lógico-aritméticas. Na implementação deste trabalho, o campo *ALUOp* foi ampliado para 3 bits, a fim de também distinguir as operações com imediato (*Tipo-I*) e a operação de repasse exigida pela instrução *lui*, sendo o detalhamento da operação final feito pela unidade de controle da ULA a partir dos campos *funct3* e *funct7*. A tabela completa, com todas as instruções implementadas e os sinais adicionais, é apresentada na Seção de Procedimentos (Tabela III).

Esses quatro grupos representam o modelo de referência apresentado em sala [2]. A mesma estrutura combinacional foi utilizada para reconhecer também os demais opcodes do conjunto de instruções adotado no projeto, atribuindo a cada um deles a combinação adequada de sinais conforme o formato correspondente. Para opcodes não previstos, todos os sinais de habilitação são mantidos desligados por padrão, evitando que uma instrução inválida cause efeitos indesejados no restante do caminho de dados.

I. Multiplexador 2:1

O multiplexador 2 para 1 é o elemento combinacional mais simples utilizado no caminho de dados: ele recebe duas entradas de dados e, dependendo do valor de um único bit de seleção, decide qual delas aparece na saída [2]. A Tabela II resume seu comportamento.

Em vez de montar um circuito seletor diferente para cada ponto do caminho de dados onde havia essa necessidade,

TABLE II
TABELA VERDADE DO MULTIPLEXADOR 2:1

Seleção	Saída
0	Entrada 0
1	Entrada 1

optou-se por um único módulo genérico e parametrizável quanto à largura do barramento, depois instanciado nos diversos pontos do caminho de dados Uniciclo em que existe seleção entre duas fontes de dados. O primeiro deles escolhe entre o dado lido do segundo registrador e o valor imediato, de acordo com o sinal `ALUSrc`. Outro multiplexador escolhe entre o conteúdo de `rs1` e o valor do PC como primeiro operando da ULA, em apoio à instrução `auipc`. No estágio de escrita de volta, dois multiplexadores em cascata selecionam entre o resultado da ULA, o dado lido da memória e o endereço de retorno `PC + 4`, de acordo com o sinal `MemtoReg` de 2 bits. Por fim, no estágio de atualização do PC, multiplexadores em cascata escolhem entre o endereço sequencial `PC + 4`, o endereço de desvio `PC + imediato` e o endereço de `jalr`, conforme a combinação dos sinais `Branch/Zero`, `Jump` e `JumpR`.

IV. PROCEDIMENTOS E MÉTODOS

A. Ferramentas Utilizadas

O desenvolvimento foi realizado no ambiente Quartus II Altera, utilizando descrição de *hardware* em linguagem Verilog para todos os módulos do caminho de dados. A verificação funcional foi conduzida no módulo de simulação Waveform Editor (Vector Waveform Editor) do próprio Quartus, complementada, em alguns módulos, por *testbenches* escritos em Verilog que comparam automaticamente a saída obtida com o valor esperado em cada caso de teste. As memórias de instruções e de dados foram inicializadas a partir dos arquivos no formato MIF (`UnicicloInst.mif` e `UnicicloData.mif`), de acordo com o mapa de endereços do RISC-V32I sugerido na especificação. A metodologia adotada foi incremental: cada módulo foi descrito e validado isoladamente antes de ser integrado ao módulo de topo (`riscv_cpu`), o que reduziu o espaço de busca durante a depuração da unidade operativa completa.

B. Contador de Programa e Memória de Instruções

O contador de programa foi descrito como um registrador de 32 bits sensível à borda de subida do *clock*, com *reset* assíncrono que o inicializa em `0x00004000`. A cada ciclo, o PC recebe o endereço calculado pela lógica de seleção do próximo PC. A memória de instruções foi descrita como um vetor de palavras de 32 bits inicializado pelo arquivo MIF. A leitura é combinacional: a partir do endereço de *byte* fornecido pelo PC, calcula-se o índice de palavra subtraindo o endereço base e deslocando dois bits à direita. Implementou-se ainda uma verificação de faixa (*valid_addr*): caso o endereço esteja abaixo do início do segmento de texto ou além do tamanho

da memória, a saída é forçada para a instrução sentinela `0x0000006F` (equivalente a `jal x0, 0`), que mantém o PC preso na posição corrente em vez de buscar uma palavra não inicializada. A motivação dessa decisão de projeto é discutida na Seção VI-A.

C. Banco de Registradores

O banco foi descrito como um vetor de 32 posições de 32 bits, com leitura combinacional nas portas `rs1` e `rs2` e escrita síncrona na borda de subida do *clock*, condicionada ao sinal `RegWrite` e ao fato de o registrador de destino não ser `x0`. Tanto na leitura quanto na escrita, `x0` é tratado como constante igual a zero. Para atender ao requisito de visualização, o conteúdo de cada registrador foi também conectado a saídas individuais nomeadas conforme a convenção ABI do RISC-V (`ra`, `sp`, `gp`, `tp`, `t0-t6`, `s0-s11`, `a0-a7`), permitindo acompanhar a janela completa de registradores durante a simulação.

D. Unidade Lógica e Aritmética e seu Controle

A ULA foi descrita como um bloco combinacional cujo comportamento é selecionado por um código de 4 bits. Além das operações aritmético-lógicas usuais (soma, subtração, AND, OR, XOR, deslocamentos e comparação com sinal), foi incluída uma operação de repasse do segundo operando (`PASSA-B`), utilizada pela instrução `lui`. A unidade de controle da ULA foi descrita de forma hierárquica: a partir do sinal `ALUOp` (3 bits), seleciona-se a categoria da operação; quando necessário, os campos *funct3* e o bit 30 da instrução refinam a operação final. Em particular, distinguiu-se o caso das instruções com imediato (Tipo-I), nas quais *funct3* = `000` deve sempre resultar em soma (`addi`), do caso Tipo-R, no qual o bit 30 diferencia `add` de `sub`. Ambos os módulos foram validados isoladamente, aplicando-se todas as combinações relevantes de `ALUOp`, *funct3* e bit 30 e conferindo o código de operação e o resultado produzidos.

E. Somador

Os somadores foram descritos como blocos combinacionais de 32 bits e instanciados duas vezes no caminho de dados, para o cálculo de `PC + 4` e de `PC + imediato`. A validação consistiu em verificar, para diferentes valores de PC e de imediato, se a saída correspondia exatamente à soma esperada, incluindo casos com imediato negativo (desvios para trás), nos quais a extensão de sinal do imediato é essencial para o cálculo correto do endereço.

F. Gerador de Imediato

O gerador de imediato foi descrito como um bloco combinacional que seleciona o formato (I, S, B, U ou J) a partir do opcode e monta o valor de 32 bits por concatenação dos campos correspondentes, com extensão de sinal. Tomou-se cuidado especial para que cada concatenação resultasse em exatamente 32 bits e para que os formatos de desvio (B) e de salto (J) reintroduzissem o bit menos significativo igual a zero. A verificação foi feita aplicando-se instruções representativas de

cada formato e conferindo o imediato resultante, com atenção particular aos casos de sinal negativo e aos deslocamentos de *beq/bne* e *jal*.

G. Memória de Dados

A memória de dados foi descrita como um vetor de palavras de 32 bits inicializado pelo arquivo MIF, endereçado pelo resultado da ULA. A escrita ocorre na borda de subida do *clock*, condicionada ao sinal *MemWrite*, e a leitura é capturada na borda de descida, condicionada implicitamente ao uso do dado pela instrução. Essa separação temporal entre escrita e leitura dentro do mesmo ciclo foi adotada para manter a coerência da execução Uniciclo, evitando condições de corrida entre as duas operações.

H. Unidade de Controle Principal

A unidade de controle principal foi descrita como um bloco de lógica combinacional, utilizando uma estrutura de decisão sobre o campo *opcode* de 7 bits extraído diretamente da instrução lida da memória. Para cada *opcode* suportado, foi definido um conjunto fixo de valores para os sinais de saída (*RegWrite*, *MemRead*, *MemWrite*, *MemtoReg*, *ALUSrc*, *Branch*, *ALUOp*, *Jump*, *JumpR* e *PCtoALU*), seguindo a tabela verdade completa apresentada na Tabela III. Para *opcodes* não previstos, todos os sinais de habilitação foram mantidos desligados por padrão, evitando que uma instrução inválida cause algum efeito indesejado no restante do caminho de dados.

A validação desse módulo foi feita de forma isolada no Waveform Editor do Quartus, aplicando-se cada *opcode* como entrada e conferindo, um a um, se a combinação de sinais gerada na saída correspondia exatamente à tabela de projeto antes da integração com o restante do caminho de dados.

I. Multiplexador 2:1

O módulo do multiplexador foi escrito de forma genérica, com a largura do barramento de dados como parâmetro, o que permitiu reaproveitá-lo nos diversos pontos de seleção do caminho de dados sem reescrever a lógica em cada lugar. Essa escolha reduz a duplicação de código e facilita a verificação, já que o comportamento do multiplexador só precisa ser validado uma vez para que a confiança se estenda a todas as instâncias seguintes. A validação foi feita no Waveform Editor, alternando o sinal de seleção entre 0 e 1 para diferentes combinações de entrada e conferindo se a entrada correta era propagada para a saída em cada caso.

J. Integração e Lógica de Salto

A integração dos módulos foi feita no módulo de topo, conectando os sinais de controle às entradas de seleção dos multiplexadores. Para suportar as instruções de salto e de formação de constantes, três pontos do caminho de dados foram acrescentados em relação ao modelo de referência: (i) um multiplexador que seleciona o primeiro operando da ULA entre *rs1* e o PC, controlado por *PCtoALU* e usado pela instrução *auipc*; (ii) um segundo nível no multiplexador de

escrita de volta, que permite gravar $PC + 4$ no registrador de destino para *jal* e *jalr*; e (iii) uma cascata de multiplexadores na atualização do PC, que seleciona entre $PC + 4$, o endereço de desvio $PC + imediato$ (compartilhado por *beq*, *bne* e *jal*) e o endereço de *jalr*, este último obtido de $(rs1 + imediato)$ com o bit menos significativo forçado a zero. A decisão de desvio condicional foi isolada em um módulo dedicado (*BranchControl*), que combina o sinal *Branch*, a saída *Zero* da ULA e o campo *funct3* para distinguir *beq* (salta se igual) de *bne* (salta se diferente).

V. RESULTADOS EXPERIMENTAIS

A. Contador de Programa e Memória de Instruções

Na simulação, verificou-se que, após o *reset*, o PC parte de $0x00004000$ e avança de quatro em quatro *bytes* a cada ciclo, exceto quando uma instrução de desvio ou salto altera o fluxo. A memória de instruções entregou, a cada endereço, a palavra de 32 bits correspondente do arquivo *UnicicloInst.mif*, e a verificação de faixa comportou-se conforme o esperado: endereços fora do segmento de texto resultaram na instrução sentinela $0x0000006F$, mantendo o PC estável em vez de buscar conteúdo indefinido.

B. Banco de Registradores

A simulação confirmou a leitura assíncrona nas portas *rs1* e *rs2* e a escrita síncrona em *rd*, com $\times 0$ permanecendo em zero em todas as situações. A exposição individual dos 32 registradores permitiu acompanhar, lado a lado, o efeito de cada instrução sobre o estado da máquina, facilitando a conferência dos resultados das demais unidades.

C. Unidade Lógica e Aritmética e seu Controle

A ULA e sua unidade de controle produziram, para cada combinação de *ALUOp*, *funct3* e bit 30, o código de operação e o resultado esperados, incluindo o caso especial *PASSA-B* usado pela instrução *lui* e a distinção entre *addi* (soma forçada) e o par *add/sub*. O sinal *Zero* acompanhou corretamente o resultado nulo, dando suporte à decisão de desvio condicional.

D. Somador

Os somadores produziram $PC + 4$ e $PC + imediato$ corretamente, inclusive para immediatos negativos, situação em que o desvio para trás depende da extensão de sinal correta do imediato.

E. Gerador de Imediato

O gerador de imediato reproduziu os valores esperados para os formatos I, S, B, U e J, com a extensão de sinal correta e com o bit menos significativo igual a zero nos formatos B e J. A conferência desses valores foi determinante para validar os endereços de desvio e de salto, conforme discutido na Seção VI-A.

TABLE III
TABELA VERDADE COMPLETA DA UNIDADE DE CONTROLE IMPLEMENTADA

Instr.	Opcode	ALUSrc	Mem2Reg	RegWrite	MemRead	MemWrite	Branch	Jump	JumpR	PCtoALU	ALUOp
Tipo-R	0110011	0	00	1	0	0	0	0	0	0	010
Tipo-I (addi, andi, ...)	0010011	1	00	1	0	0	0	0	0	0	011
lw / lhu	0000011	1	01	1	1	0	0	0	0	0	000
sw	0100011	1	XX	0	0	1	0	0	0	0	000
beq / bne	1100011	0	XX	0	0	0	1	0	0	0	001
jal	1101111	X	10	1	0	0	0	1	0	0	000
jalr	1100111	1	10	1	0	0	0	0	1	0	000
lui	0110111	1	00	1	0	0	0	0	0	0	100
auipc	0010111	1	00	1	0	0	0	0	0	1	000
inválido (default)	—	0	00	0	0	0	0	0	0	0	000

TABLE IV
CASOS DE TESTE DA UNIDADE DE CONTROLE

Opcode (bin)	Opcode (hex)	Resultado
0110011	33	PASS
0010011	13	PASS
0000011	03	PASS
0100011	23	PASS
1100011	63	PASS
1101111	6F	PASS
1100111	67	PASS
0110111	37	PASS
0010111	17	PASS
1111111	7F	PASS

F. Memória de Dados

A memória de dados respondeu corretamente às instruções *lw* e *sw*: a escrita na borda de subida e a leitura na borda de descida permitiram que, dentro de um mesmo ciclo, o dado armazenado fosse posteriormente recuperado sem condição de corrida, mantendo a coerência da execução Unicidade.

G. Unidade de Controle Principal e Multiplexador 2:1

A verificação de ambos os módulos foi feita por meio de *testbenches* próprios, escritos em Verilog e executados no simulador, que comparam automaticamente a saída obtida com o valor esperado em cada caso de teste e reportam o resultado.

Para a unidade de controle, o *testbench* aplicou, um a um, os opcodes correspondentes a todos os grupos de instruções suportados pelo projeto, além de um opcode inválido para verificar o comportamento padrão. A Tabela IV resume os casos testados, todos aprovados.

Como a unidade de controle é um circuito puramente combinacional, a forma de onda gerada na simulação apresenta os sinais de saída constantes para cada opcode aplicado, sem transições intermediárias, o que é o comportamento esperado para esse tipo de lógica.

Para o multiplexador, o *testbench* aplicou diferentes combinações de dados de 32 bits nas duas entradas, variando o sinal de seleção entre 0 e 1, e conferiu se a saída reproduzia exatamente a entrada selecionada em cada caso. Foram usados valores representativos como AAAAAAAA, 55555555, DEADBEEF e CAFEBABE, e todos os casos foram aprovados, confirmando o funcionamento correto do módulo para qualquer dado de entrada.

H. Análise de Desempenho das Unidades Funcionais

A análise de desempenho deve ser obtida no modo de compilação *Timing* do Quartus, que considera os atrasos de propagação reais de cada unidade funcional. Qualitativamente, espera-se que o caminho crítico do processador Unicidade seja determinado pela instrução de carga (*lw*), cujo percurso atravessa, em sequência, a memória de instruções, o banco de registradores, o gerador de imediato, a ULA e a memória de dados, antes da escrita de volta no banco de registradores. As instruções de salto (*jal*, *jalr*) e de formação de constantes (*lui*, *auipc*) tendem a apresentar caminhos mais curtos, por não acessarem a memória de dados. A Tabela de tempos por módulo (valor absoluto em submúltiplos de segundo e percentual do ciclo) e a estimativa do período de *clock* devem ser preenchidas a partir dos relatórios de temporização da síntese, identificando os pontos de *gargalo* (*bottlenecks*).

VI. ANÁLISE DE RESULTADOS

A unidade de controle principal e o multiplexador atenderam ao comportamento esperado em todos os casos de teste, gerando os sinais corretos para cada opcode e propagando o dado certo em função do sinal de seleção. A verificação automática por *testbench* foi importante porque permitiu confirmar o comportamento de cada módulo de forma isolada, antes da integração com o restante do caminho de dados, reduzindo a chance de erros difíceis de localizar em uma etapa posterior. Após a integração, o caminho de dados completo executou o conjunto de instruções pré-compiladas, atualizando corretamente o PC, o banco de registradores e a memória de dados ao longo dos ciclos.

A. Dificuldades de Implementação e Depuração

As principais dificuldades do trabalho concentraram-se em dois eixos relacionados: (i) a implementação das instruções de salto e de formação de constantes (*jal*, *jalr*, *lui* e *auipc*), que extrapolam o caminho de dados de referência; e (ii) o tratamento de saltos para fora da região válida da memória de instruções, agravado por imediatos que inicialmente não eram montados corretamente em 32 bits.

1) *Implementação dos saltos, auipc, lui, jal e jalr*: O caminho de dados Unicidade de referência [2] contempla apenas as instruções Tipo-R, *lw*, *sw* e *beq*. A inclusão das

demais instruções exigiu acrescentar sinais de controle e caminhos de dados que não existem na Figura 1 da especificação, o que foi a fonte da maior parte das dificuldades:

- **Endereço de retorno (jal/jalr).** Essas instruções precisam gravar $PC + 4$ no registrador de destino, mas o multiplexador de escrita de volta original só escolhia entre o resultado da ULA e o dado da memória. Foi necessário ampliar o sinal `MemtoReg` para 2 bits e acrescentar um segundo nível de multiplexador para rotear $PC + 4$ até o banco de registradores. Antes dessa correção, o registrador de destino recebia o resultado da ULA, e os endereços de retorno gravados estavam errados, o que só se manifestava ciclos depois, ao retornar do salto.
- **auipc — PC como operando da ULA.** A instrução `auipc` calcula $PC + \text{imediato}$, e não $rs1 + \text{imediato}$. Como a ULA, no modelo de referência, recebe sempre `rs1` em seu primeiro operando, foi preciso introduzir o sinal `PcttoALU` e um multiplexador adicional que substitui `rs1` pelo PC nessa instrução.
- **lui — repasse do imediato.** A instrução `lui` deveria simplesmente escrever o imediato (com os 12 bits inferiores zerados) no registrador de destino. Uma armadilha sutil é que os bits [19:15] da instrução `lui` fazem parte do imediato, mas o caminho de dados sempre os interpreta como o endereço de `rs1`; assim, somar `rs1` ao imediato produziria um resultado contaminado por uma leitura “fantasma” de registrador. A solução foi criar uma operação dedicada na ULA (`PASSA-B`, código `4'b1000`), selecionada por `ALUOp = 100`, que descarta o primeiro operando e repassa apenas o imediato. O mesmo cuidado vale para `auipc`, em que a leitura fantasma de `rs1` é neutralizada pelo multiplexador de `PcttoALU`.
- **Cascata de seleção do próximo PC.** Com três origens possíveis para o próximo endereço ($PC + 4$; $PC + \text{imediato}$ para `beq/bne/jal`; e o alvo de `jalr`), foi necessário organizar uma cascata de multiplexadores com a prioridade correta. O primeiro nível escolhe entre $PC + 4$ e $PC + \text{imediato}$ a partir de $(PcSrc \text{ OR } Jump)$; o segundo nível sobrepõe o alvo de `jalr` quando `JumpR` está ativo. Erros na ordem dessa cascata faziam, por exemplo, um `jalr` ser “mascarado” por um desvio condicional inativo.
- **jalr — alinhamento do alvo.** O endereço de destino do `jalr` é $(rs1 + \text{imediato})$ com o bit menos significativo forçado a zero ($\{alu_result[31:1], 1'b0\}$). Omitir esse zeramento gerava endereços desalinhados, que por sua vez caíam fora das posições válidas da memória de instruções.
- **Distinção jal × jalr no gerador de imediato.** Embora ambas sejam saltos incondicionais, `jal` usa imediato no formato J (relativo ao PC) e `jalr` usa imediato no formato I (relativo a `rs1`, com `ALUSrc = 1`). Tratar as duas com o mesmo formato de imediato levava a alvos completamente incorretos.

2) Saltos para fora da memória e validação dos 32 bits:

A segunda grande dificuldade surgiu justamente quando os saltos passaram a funcionar: em algumas execuções, o PC alcançava endereços fora da região efetivamente carregada do segmento de texto. Como, inicialmente, a memória de instruções não validava o endereço recebido, a leitura de uma posição não inicializada produzia uma palavra de 32 bits indefinida (valores X na simulação). Essa palavra indefinida era então decodificada pela unidade de controle como um opcode arbitrário, ativando sinais de controle aleatórios e corrompendo o banco de registradores e a memória de dados. O sintoma observado — registradores assumindo valores sem sentido vários ciclos após o salto — era difícil de localizar, pois a causa (o salto para fora da memória) e o efeito (a corrupção do estado) estavam separados no tempo.

A investigação revelou duas causas que se reforçavam mutuamente:

- 1) **Tradução de endereço ausente.** Como o PC inicia em `0x00004000` e não em zero, o índice de palavra não pode ser obtido diretamente de `addr >> 2`; é preciso primeiro subtrair o endereço base do segmento (`byte_offset = addr - TEXT_BASE`) e só então deslocar dois bits à direita. Sem essa tradução, qualquer endereço já indexava posições muito além do conteúdo carregado.
- 2) **Imediatos não validados em 32 bits.** O código de montagem dos imediatos, em versões iniciais, não garantia que cada concatenação somasse exatamente 32 bits — seja por erro na largura da replicação do bit de sinal, seja por omissão do bit menos significativo implícito (igual a zero) nos formatos B e J. Um imediato mal formado produzia um *offset* de desvio/salto incorreto e, por consequência, fazia o desvio aterrissar fora da região correta da memória. Esse é o elo direto entre o problema da “validação dos 32 bits” e os “saltos para fora da memória”.

A solução combinou três medidas. Primeiro, padronizou-se a tradução de endereço relativa a `TEXT_BASE`. Segundo, reescreveu-se o gerador de imediato verificando, campo a campo, que cada formato soma exatamente 32 bits: por exemplo, o formato B é montado como $20 + 1 + 6 + 4 + 1 = 32$ bits e o formato J como $12 + 8 + 1 + 10 + 1 = 32$ bits, sempre com o bit menos significativo igual a zero. Terceiro, adicionou-se à memória de instruções uma verificação de faixa (`valid_addr`): quando o endereço está abaixo do início do segmento de texto ou além do tamanho da memória, a saída é forçada para a instrução sentinela `0x0000006F (jal x0, 0)`, que prende o PC na posição corrente. Essa instrução sentinela transforma um salto para fora da memória — antes catastrófico e silencioso — em uma condição controlada e observável: em vez de propagar valores indefinidos, a máquina entra em um laço estável, tornando o problema imediatamente visível na simulação.

A metodologia de depuração que permitiu isolar essas falhas baseou-se na verificação modular (cada bloco testado

isoladamente antes da integração) e na observação, ciclo a ciclo, do PC e da palavra de instrução de 32 bits no Waveform Editor. A introdução da instrução sentinela foi particularmente útil porque deu uma “assinatura” reconhecível ao evento de salto para fora da memória, encurtando o tempo entre a manifestação do sintoma e a identificação da causa.

VII. CONCLUSÃO

A construção da Unidade Operativa Uniciclo RISC-V32I mostrou, na prática, como a decodificação do opcode pode ser isolada em um bloco puramente combinacional, como um único componente seletor genérico pode ser reaproveitado em vários pontos do caminho de dados e como a separação física entre memórias de instrução e de dados (organização Harvard) viabiliza a busca e o acesso a dados em um único ciclo. Essa organização modular deixou o projeto mais claro e facilitou a verificação individual de cada parte antes da integração da unidade operativa completa.

As maiores dificuldades não estiveram nos módulos do modelo de referência, mas na extensão necessária para suportar as instruções de salto e de formação de constantes (`jal`, `jalr`, `lui` e `auipc`), que demandaram novos sinais de controle (`Jump`, `JumpR`, `PCToALU`, `MemtoReg` de 2 bits), uma nova operação na ULA (`PASSA-B`) e uma cascata de multiplexadores para a seleção do próximo PC. O caso mais instrutivo foi o dos saltos para fora da memória de instruções: ele evidenciou como um imediato mal validado em 32 bits e a ausência de tradução e validação de endereço podem se combinar em uma falha de difícil diagnóstico, e como uma decisão de projeto simples — retornar uma instrução sentinela bem definida para endereços inválidos — pode converter uma falha silenciosa em um comportamento estável e observável. O laboratório reforçou, assim, tanto os conceitos de organização e arquitetura de computadores quanto a importância de uma metodologia de verificação incremental e de decisões de projeto defensivas.

AGRADECIMENTOS

REFERENCES

- [1] F. Vidal, *RISC-V Reference Guide v25*, 2025.
- [2] D. A. Patterson e J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2020.